PRAKTIKUM
SISTEM BASIS DATA

Nama	**Muhammad Bryan Farras**	**No. Modul**	**6**
NPM	**2306230975**	**Tipe**	**TUTAM**

1. Melanjutkan Backend.
2. Response.

Method	Endpoint	Success	Failed
GET	/transaction	{ "success": true, "message": "Transactions Found", "payload": [{ "id": "9eedd229-cd6e-4cec-9e4a-20f636cada3f", "user_id": "fac9e6d6-e6c1-4568-a546-b10c534680c8", "item_id": "6d26d61e-4cd2-4de1-a751-378685ba6920", "quantity": 1, "total": 50000, "status": "pending", "created_at": "2025-04-13T07:08:07.126Z", "user": { "id": "fac9e6d6-e6c1-4568-a546-b10c534680c8", "name": "Bryan", "email": "bryan@gmail.com", "password": "cisco", "balance": 100000, "created_at": "2025-03-12T05:26:03.5507" }, "item": { "id": "6d26d61e-4cd2-4de1-a751-378685ba6920", "name": "lanyard", "price": 50000, "store_id": "19ecfed6-1992-47b4-a63e-9b6b51bded4e", "image_url": "https://res.cloudinary.com/dqbpbpryj/ "stock": 5, "created_at": "2025-03-16T15:23:44.548282" } }]	<!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8"> <title>Error</title> </head> <body> <pre>Cannot POST /transaction</pre> </body> </html>

3. Meningkatkan kode untuk scalability, Security, dan error handling.
4. Hasil Peningkatan

Tipe	Implementasi	Deskripsi
Skalabilitas & performa	`function startServer() { const express = require('express'); const bodyParser = require('body-parser'); const cors = require('cors'); require('dotenv').config(); const app = express(); const PORT = process.env.PORT || 3000; const corsOptions = { origin: '*',`	Pada code express awal di index.js, semua kode tersebut dijadikan dalam sebuah function yang akan digunakan pada pm2. Hal ini memungkinkan eksekusi dari express menjadi multicore dan performa menjadi lebih

	methods: ['GET', 'POST', 'PUT', 'DELETE'],
	allowedHeaders: ['Content-Type', 'Authorization']
	};
	
	app.use(cors(corsOptions));
	app.use(bodyParser.json());
	app.use(bodyParser.urlencoded({ extended: true }));
	app.use(express.json());
	
	app.use('/store', require('./src/routes/store.route'));
	app.use('/user', require('./src/routes/user.route'));
	app.use('/item', require('./src/routes/item.route'));
	app.use('/transaction', require('./src/routes/transaction.route'));
	
	app.listen(PORT, () => {
	console.log(`Worker ${process.pid} running on port ${PORT}`);
	});
	}
	
	const cluster = require('cluster');
	const os = require('os');
	
	const numCPUs = os.cpus().length;
	
	if (cluster.isMaster) {
	console.log(`Master ${process.pid} is running`);
	
	for (let i = 0; i < numCPUs; i++) {
	cluster.fork();
	}
	
	cluster.on('exit', (worker) => {
	console.log(`Worker ${worker.process.pid} died`);
	cluster.fork();
	});
	
	} else {
	startServer();
	}
		cepat. Contoh dari penggunaan ini adalah penanganan request secara paralel. PM2 merupakan mekanisme load balancing, yang artinya adalah semua instance akan diberikan tugas yang relatif rata secara otomatis. Bila terdapat satu instance atau worker yang gagal, instance tersebut dapat digantikan langsung dengan instance pengganti.

Keamanan	app.use(helmet());
	app.use(helmet.frameguard({ action: 'deny' }));
	app.use(helmet.hidePoweredBy());
	app.use(helmet.xssFilter());
	app.use(helmet.noSniff());
	app.use(helmet.contentSecurityPolicy({
	directives: {
		Menggunakan helmet karena kemampuannya untuk melindungi aplikasi dari berbagai serangan umum yang sering terjadi di dunia web, seperti Cross-Site Scripting (XSS), Clickjacking, dan MIME-type

	defaultSrc: ["'self'"],
	scriptSrc: ["'self'", "trusted-cdn.com"],
	styleSrc: ["'self'", "trusted-styles.com"],
	},
	}));
	
	app.use(helmet.strictTransportSecurity({
	maxAge: 31536000,
	includeSubDomains: true,
	preload: true,
	}));
	app.use(helmet.referrerPolicy({ policy: 'no-referrer' }));
	app.use(helmet.dnsPrefetchControl({ allow: false }));
		sniffing. Dengan mengaktifkan helmet, aplikasi otomatis dilindungi dengan menambahkan header keamanan yang mencegah serangan-serangan ini. Misalnya, dengan header seperti X-Frame-Options, yang mencegah aplikasi kamu dimasukkan dalam frame (untuk melawan clickjacking), atau Strict-Transport-Security yang memastikan aplikasi hanya bisa diakses melalui HTTPS.

Error Handling	Tidak ada.	Error handling sudah cukup baik. Pada semua menggunakan mekanisme try catch yang membuat kode lebih mudah dimengerti dan semua data error diberikan oleh base response. Try berarti program menjalankan tugas utamanya, dan bila terdapat error maka akan melakukan catch untuk menangani masalah tersebut.